ADVANCED OBJECT ORIENTED PROGRAMMING (25CS2205E)

Practical Exercise 1: Bank Account and Money Transfers

Name: P Sreshta
Roll No: 2520030217
Sec: 4
Experiment: Practical Exercise 1: Bank Account and Money Transfers

1. Aim

To analyse the Bank Account and Money Transfers case study, identify the design problems and business invariants, and develop a Java banking application in which objects protect their own rules (encapsulation, validation, immutability), so that overdraft limits, account states, transaction amounts, transfers and the ledger always remain valid and consistent.

2. Software Used

Software / Tool	Details
Java	JDK 21 (compile and run the application)
IDE / Editor	VS Code / IntelliJ IDEA (edit to match what you used)
Operating System	macOS
Version Control	Git and GitHub (code and PDF submission)

3. Case Study Summary

A retail bank wants a Java application that lets customers maintain accounts, deposit, withdraw and transfer funds. The development team's first version used public fields, primitive data types and separate service methods. The bank reported five operational problems, which are analysed below.

3.1 Design problems identified (Message 1 of the task)

Problem reported	Root cause in the basic design	How the improved design prevents it
Withdrawals beyond overdraft limit	Balance is a public field; the service method may forget to check the limit	`withdraw()` checks `requireWithinOverdraft()`; balance is private
Money deducted but not credited when a failure occurs	Debit and credit are two separate, unprotected steps	`FundTransfer` validates first, then debits and credits, and rolls back with a `REVERSAL` entry if anything fails

Problem reported	Root cause in the basic design	How the improved design prevents it
Frozen accounts accept withdrawals	Status is not part of the object; the check is optional	`AccountStatus` enum; `withdraw()` calls `requireActive()` so FROZEN and CLOSED are rejected
Balance does not match transaction history	Balance and history are updated by different code at different times	Only `credit()` and `debit()` change the balance, and both add the `LedgerEntry` in the same step; `isLedgerConsistent()` verifies it
Zero or negative amounts accepted	Primitive types (`double`) accept any value	`Money` value object plus `requirePositive()` on every operation

Other design problems found:

- Public fields let any class change balance or status directly, bypassing all rules.
- `double`/primitives for money cause rounding errors and carry no currency; `Money` stores paise as `long` plus currency.
- Rules are scattered across service methods, so every new method must remember every rule.
- No state model: nothing says which operations are legal in ACTIVE, FROZEN or CLOSED.
- History can be edited or deleted, so there is no reliable audit trail.
- Concurrent transfers can interleave and corrupt data; `transfer()` is synchronized.

4. Activity 1: Analyse the Banking Problem (Modelling)

4.1 Q1. Domain invariants

#	Business rule (invariant)	Why it is important	Consequence if violated
1	Every transaction amount must be greater than zero	Zero or negative amounts make no business sense and can be abused to reverse a transaction's direction	A negative deposit acts as a hidden withdrawal; balances become wrong; possible fraud
2	Balance must never fall below minus the overdraft limit	The overdraft limit is the credit the bank has agreed to give	Bank gives unauthorised credit and loses money
3	A FROZEN or CLOSED account cannot be withdrawn from	Frozen accounts are blocked for legal or security reasons	Money leaves a blocked account; legal and fraud risk
4	A CLOSED account accepts no transactions, cannot be unfrozen, and can be closed only with zero balance	Closing is the end of the account lifecycle	Money lost inside a dead account; transactions on non-existent accounts
5	Account balance must always equal the sum of its ledger entries	The ledger is the proof of every rupee in the account	Disputes, failed audits, customers lose trust

#	Business rule (invariant)	Why it is important	Consequence if violated
6	Ledger entries are immutable: never modified or deleted	An unchangeable record is the audit trail	History can be tampered with; fraud cannot be traced
7	A transfer is atomic: the amount debited from the source equals the amount credited to the destination, or nothing changes	Money must be conserved in the system	Money disappears or is created; customer loses funds
8	Account number is mandatory, unique and never changes	It is the identity of the account	Transactions posted to the wrong account
9	Money values can be combined only if they have the same currency	Adding INR to USD without conversion is meaningless	Wrong totals and incorrect balances

4.2 Q2. Entities and Value Objects

Concept	Classification	Identity and equality	Lifecycle and mutability
Bank Account	Entity	Has identity (account number). Two accounts with the same balance are still different accounts; equality is by identity.	Created, then moves ACTIVE, FROZEN, CLOSED. Mutable: balance and status change over time.
Account Number	Value Object	Just an identifier value with no identity of its own; two numbers are equal if the text is equal.	Fixed for life of the account (final). Immutable.
Money	Value Object	Equality by value (amount and currency). Rs 500 is the same as any other Rs 500.	No lifecycle. Immutable: `add()` and `subtract()` return a new Money.
Ledger Entry	Entity (immutable record)	Has a unique transaction ID. Two deposits of the same amount at the same time are still different records.	Created once, never modified or deleted. Corrections are made by adding a new entry.
Overdraft Limit	Value Object	Described only by its amount (a Money value); equal if the amounts are equal.	Set when the account is created (final). To change it, replace it with a new value.
Fund Transfer	Entity	Has identity (transfer ID); each transfer is a unique business event even if amounts match.	Has a lifecycle: initiated, debited, then completed or rolled back.

4.3 Account attributes, behaviours and valid states

Class	Attributes (what it knows)	Behaviours (what it does)
BankAccount	`accountNumber, balance, overdraftLimit, status, ledger`	`deposit(), withdraw(), freeze(), unfreeze(), close(), getBalance(), isLedgerConsistent()`
Money	`amountInPaise, currency`	`add(), subtract(), isPositive(), compareTo()`
LedgerEntry	`transactionId, transactionType, amount, balanceAfter, timestamp`	Getters only (read-only)
AccountStatus	`ACTIVE, FROZEN, CLOSED`	Defines the allowed account states
FundTransfer	`transferId, source, destination, amount`	`transfer()`

Valid and invalid operations by account state:

State	Deposit	Withdraw	Transfer out	Transfer in	Freeze	Unfreeze	Close
ACTIVE	Yes	Yes (within overdraft)	Yes	Yes	Yes	No	Only if balance is zero
FROZEN	Yes	No	No	Yes	Yes	Yes	Only if balance is zero
CLOSED	No	No	No	No	No	No	No

5. Activity 2: Design Self-Enforcing Objects

5.1 Class design (Project Structure)

Class	Role in the design
Money	Immutable value object (paise stored as `long`, plus currency). Rejects null/blank currency, prevents currency mixing.
AccountStatus	Enum with `ACTIVE, FROZEN, CLOSED` to model the account lifecycle.
BankAccount	Entity. Private fields, validates every operation, writes a ledger entry for every change, exposes only a read-only ledger view.
LedgerEntry	Immutable record of one transaction (`final` fields, getters only).
FundTransfer	Atomic transfer: validate both accounts, debit, credit, rollback on failure.
Main	Test driver that runs every scenario and prints the results.

5.2 Q3. How the design enforces the rules

Requirement	How the design ensures it
Invalid transaction amounts are rejected	Every operation calls `requirePositive(amount)`. Zero, negative and null amounts throw `IllegalArgumentException` before any state is changed. `LedgerEntry` also rejects non-positive amounts.
No withdrawal when frozen or closed	`withdraw()` calls `requireActive()`, which throws `IllegalStateException` if status is FROZEN or CLOSED. Status can be changed only through `freeze()`, `unfreeze()` and `close()`.
Overdraft limit is respected	`requireWithinOverdraft()` computes the new balance and rejects the withdrawal if it would fall below the negative overdraft limit. The limit is `final`, so it cannot be changed after creation.
External classes cannot modify account data	All fields are `private`. There are no setters. Balance changes only through `deposit()` and `withdraw()`. `getLedger()` returns `Collections.unmodifiableList()`, so the history cannot be altered from outside. `Money` and `LedgerEntry` are immutable (`final` fields).

Project folder structure in VS Code (6 java files and their compiled .class files):

```
EXPLORER > FILES
├── AccountStatus.class | AccountStatus.java
├── BankAccount.class | BankAccount.java
├── FundTransfer.class | FundTransfer.java
├── LedgerEntry.class | LedgerEntry.java
├── Main.class | Main.java
└── Money.class | Money.java
```

5.3 Java source code

All classes are in one folder (default package). Compile with: `javac *.java` and run with: `java Main`

`Money.java`

```
import java.util.Objects;

/**
 * Value Object: immutable, no identity, equality by value.
 * Amount is stored in paise (long) to avoid floating-point errors.
 */
public final class Money implements Comparable<Money> {
    private final long amountInPaise;
    private final String currency;

    public Money(long amountInPaise, String currency) {
        if (currency == null || currency.isBlank()) {
            throw new IllegalArgumentException("Currency is required");
        }
        this.amountInPaise = amountInPaise;
        this.currency = currency;
    }

    public static Money rupees(long rupees) {
        return new Money(rupees * 100, "INR");
    }

    public static Money zero() {
        return new Money(0, "INR");
    }

    public Money add(Money other) {
        requireSameCurrency(other);
        return new Money(Math.addExact(this.amountInPaise, other.amountInPaise), currency);
    }

    public Money subtract(Money other) {
        requireSameCurrency(other);
        return new Money(Math.subtractExact(this.amountInPaise, other.amountInPaise),
currency);
    }

    public Money negate() {
        return new Money(-amountInPaise, currency);
    }

    public boolean isPositive() {
        return amountInPaise > 0;
    }

    public boolean isNegative() {
        return amountInPaise < 0;
    }

    public long getAmountInPaise() {
        return amountInPaise;
    }

    public String getCurrency() {
        return currency;
    }

    private void requireSameCurrency(Money other) {
        if (other == null) {
```

AccountStatus.java

```
public enum AccountStatus {
    ACTIVE,
    FROZEN,
    CLOSED
}
```

BankAccount.java

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * Entity: has identity (accountNumber) and a lifecycle (ACTIVE -> FROZEN -> CLOSED).
 * Self-enforcing: every rule is checked inside the object; fields are private.
 */
public class BankAccount {
    // Ledger transaction types
    public static final String DEPOSIT      = "DEPOSIT";
    public static final String WITHDRAWAL   = "WITHDRAWAL";
    public static final String TRANSFER_IN  = "TRANSFER_IN";
    public static final String TRANSFER_OUT = "TRANSFER_OUT";
    public static final String REVERSAL     = "REVERSAL";

    private final String accountNumber;
    private Money balance;
    private final Money overdraftLimit;
    private AccountStatus status;
    private final List<LedgerEntry> ledger = new ArrayList<>();

    public BankAccount(String accountNumber, Money overdraftLimit) {
        if (accountNumber == null || accountNumber.isBlank()) {
            throw new IllegalArgumentException("Account number is required");
        }
        if (overdraftLimit == null || overdraftLimit.isNegative()) {
            throw new IllegalArgumentException("Overdraft limit cannot be negative");
        }
        this.accountNumber = accountNumber;
        this.overdraftLimit = overdraftLimit;
        this.balance = Money.zero();
        this.status = AccountStatus.ACTIVE;
    }

    // Public operations
    public void deposit(Money amount) {
        requirePositive(amount);
        requireNotClosed();
        credit(amount, DEPOSIT);
    }

    public void withdraw(Money amount) {
        requirePositive(amount);
        requireActive();
        requireWithinOverdraft(amount);
        debit(amount, WITHDRAWAL);
    }

    public void freeze() {
        if (status == AccountStatus.CLOSED) {
            throw new IllegalStateException("Closed account cannot be frozen");
        }
        status = AccountStatus.FROZEN;
    }

    public void unfreeze() {
        if (status != AccountStatus.FROZEN) {
            throw new IllegalStateException("Only a FROZEN account can be unfrozen");
```

LedgerEntry.java

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.concurrent.atomic.AtomicLong;

/**
 * Entity record that is IMMUTABLE: once written it is never modified or deleted.
 * Corrections are made by adding a new (reversal) entry.
 */
public final class LedgerEntry {
    private static final AtomicLong COUNTER = new AtomicLong(1000);

    private final String transactionId;
    private final String transactionType;
    private final Money amount;
    private final Money balanceAfter;
    private final String timestamp;

    public LedgerEntry(String transactionType, Money amount, Money balanceAfter) {
        if (transactionType == null || transactionType.isBlank()) {
            throw new IllegalArgumentException("Transaction type is required");
        }
        if (amount == null || !amount.isPositive()) {
            throw new IllegalArgumentException("Ledger amount must be positive");
        }
        if (balanceAfter == null) {
            throw new IllegalArgumentException("Balance after transaction is required");
        }
        this.transactionId = "TXN" + COUNTER.incrementAndGet();
        this.transactionType = transactionType;
        this.amount = amount;
        this.balanceAfter = balanceAfter;
        this.timestamp = LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm:ss"));
    }

    public String getTransactionId()   { return transactionId; }
    public String getTransactionType() { return transactionType; }
    public Money getAmount()           { return amount; }
    public Money getBalanceAfter()     { return balanceAfter; }
    public String getTimestamp()       { return timestamp; }

    @Override
    public String toString() {
        return String.format("%s | %-12s | %-15s | Balance: %-14s | %s",
            transactionId, transactionType, amount, balanceAfter, timestamp);
    }
}
```

FundTransfer.java

```
/**
 * Atomic money transfer: either BOTH accounts change or NEITHER does.
 * Step 1: validate both sides first (fail fast, nothing changed).
 * Step 2: debit source, credit destination.
 * Step 3: if credit fails, compensate by reversing the debit (rollback).
 */
public class FundTransfer {
    private final String transferId;
    private final BankAccount source;
    private final BankAccount destination;
    private final Money amount;
    private boolean simulateCrashAfterDebit = false; // for demonstration only

    public FundTransfer(String transferId, BankAccount source, BankAccount destination,
Money amount) {
        if (transferId == null || transferId.isBlank()) {
            throw new IllegalArgumentException("Transfer ID is required");
        }
        if (source == null || destination == null) {
            throw new IllegalArgumentException("Source and destination accounts are
required");
        }
        if (source.getAccountNumber().equals(destination.getAccountNumber())) {
            throw new IllegalArgumentException("Source and destination must be different
accounts");
        }
        if (amount == null || !amount.isPositive()) {
            throw new IllegalArgumentException("Transfer amount must be greater than
zero");
        }
        this.transferId = transferId;
        this.source = source;
        this.destination = destination;
        this.amount = amount;
    }

    public void simulateCrashAfterDebit() {
        this.simulateCrashAfterDebit = true;
    }

    public synchronized void transfer() {
        // Step 1: validate both accounts BEFORE changing anything
        source.checkCanSend(amount);
        destination.checkCanReceive(amount);

        // Step 2: debit source
        source.debitForTransfer(amount);

        try {
            if (simulateCrashAfterDebit) {
                throw new RuntimeException("System crash after debit!");
            }
            // Step 3: credit destination
            destination.creditForTransfer(amount);
        } catch (RuntimeException ex) {
            // Rollback: put the money back using a REVERSAL ledger entry
            source.reverseTransferDebit(amount);
            throw new IllegalStateException(
                "Transfer " + transferId + " failed and was rolled back: " +
```

Main.java

```
public class Main {
    static void line(String title) {
        System.out.println("\n=== " + title + " ===");
    }

    static void expectError(String label, Runnable action) {
        try {
            action.run();
            System.out.println("[FAIL] " + label + " -> no error thrown");
        } catch (RuntimeException e) {
            String msg = (e.getMessage() != null) ? e.getMessage() :
e.getClass().getSimpleName();
            System.out.println("[OK]   " + label + " -> rejected: " + msg);
        }
    }

    public static void main(String[] args) {
        BankAccount a = new BankAccount("ACC1001", Money.rupees(5000)); // overdraft Rs
5000
        BankAccount b = new BankAccount("ACC1002", Money.rupees(2000));

        line("Test 1: Valid deposit");
        a.deposit(Money.rupees(20000));
        b.deposit(Money.rupees(3000));
        System.out.println(a);
        System.out.println(b);

        line("Test 2: Valid withdrawal");
        a.withdraw(Money.rupees(2000));
        System.out.println(a);

        line("Test 3: Invalid amounts (zero / negative / null)");
        expectError("Deposit 0",    () -> a.deposit(Money.rupees(0)));
        expectError("Deposit -500", () -> a.deposit(Money.rupees(-500)));
        expectError("Withdraw -100",() -> a.withdraw(Money.rupees(-100)));
        expectError("Withdraw null",() -> a.withdraw(null));

        line("Test 4: Overdraft limit");
        BankAccount c = new BankAccount("ACC1003", Money.rupees(5000));
        c.deposit(Money.rupees(1000));
        c.withdraw(Money.rupees(5000)); // balance -4000, within limit
        System.out.println("Within limit : " + c);
        expectError("Withdraw 2000 (beyond overdraft)", () ->
c.withdraw(Money.rupees(2000)));

        line("Test 5: Frozen account");
        b.freeze();
        System.out.println(b);
        expectError("Withdraw from frozen", () -> b.withdraw(Money.rupees(100)));
        b.unfreeze();
        b.withdraw(Money.rupees(100));
        System.out.println("After unfreeze : " + b);

        line("Test 6: Closed account");
        BankAccount d = new BankAccount("ACC1004", Money.rupees(0));
        d.deposit(Money.rupees(500));
        expectError("Close with balance", d::close);
        d.withdraw(Money.rupees(500));
        d.close();
```

6. Activity 3: Handle Failures and Maintain Consistency

6.1 Q4. Money transfer and aggregate consistency

Scenario: Rs 5,000 is transferred from Account A to Account B. The system deducts it from A and crashes before crediting B.

(a) Problems that can arise

- Customer A loses Rs 5,000 and B never receives it, so money has vanished from the system.
- A's ledger shows a debit with no matching credit anywhere, so the books do not balance.
- Retrying the transfer may deduct Rs 5,000 a second time.
- Customer disputes, loss of trust and audit failure.

(b) Invariants violated

- Invariant 7: the transfer is not atomic, and money is not conserved.
- Invariant 5: the books are inconsistent because a debit exists without its matching credit.

(c) Design that prevents money loss

- Validate both accounts first (amount, status, overdraft, different accounts) so that a bad transfer fails before anything changes.
- Treat debit and credit as one unit of work inside `FundTransfer.transfer()`, which is synchronized.
- Give every transfer a unique `transferId` so it can be traced and not applied twice.
- In a real system, wrap both updates in one database transaction (BEGIN, COMMIT, ROLLBACK).

(d) How failures during the transfer are handled

- If the credit fails, the catch block calls `reverseTransferDebit()`, which puts the money back with a `REVERSAL` ledger entry.
- The failed attempt stays visible in the history (`TRANSFER_OUT` followed by `REVERSAL`); nothing is deleted.
- The caller receives an exception saying the transfer failed and was rolled back.
- If the whole program crashes, a database transaction rolls back automatically, or a recovery job reads the stored transfer status (INITIATED, DEBITED) and completes or reverses it.

(e) If the accounts belong to different banking systems

- One database transaction cannot span two banks, so atomicity is not available directly.
- Use a saga with compensating transactions or two-phase commit: debit and hold the money, request the credit from the other bank, then confirm or reverse the hold.
- Use idempotency keys (the `transferId`) so retries never double-debit, and send status queries on timeout.

- Run reconciliation against the other bank's records; the system is eventually consistent. This is how real systems like NEFT, IMPS and UPI work through a clearing network.

6.2 Transfer workflow diagram

START — Transfer Rs 5,000 : A to B

Step 1: VALIDATE BEFORE CHANGING ANYTHING
Amount > 0, accounts different, A is ACTIVE, A stays within overdraft, B is not CLOSED.
If invalid: REJECT — No balance changed, no ledger entry.

Step 2: DEBIT SOURCE (when valid)
A.balance − 5,000 | Ledger: `TRANSFER_OUT`

Step 3: CREDIT DESTINATION
B.balance + 5,000 | Ledger: `TRANSFER_IN`
If credit OK: Transfer COMPLETE — Money conserved.

ROLLBACK (compensation — on crash / error after debit)
Reverse the debit on A | Ledger: `REVERSAL` entry | History kept, nothing deleted.
Result: Transfer FAILED — A and B balances restored.

6.3 Q5. Ledger management and design evaluation

(a) Information in a ledger entry

- Transaction ID (unique), transaction type (DEPOSIT, WITHDRAWAL, TRANSFER_IN, TRANSFER_OUT, REVERSAL), amount, balance after the transaction, and timestamp (implemented).
- In a production system: account number, transfer or reference ID, description, and the user or channel that initiated it.

(b) Should a completed entry be modified or deleted?

- No. The ledger is an append-only audit trail; editing or deleting entries would allow tampering and break the link between history and balance.
- Mistakes are corrected by adding a new compensating entry (such as REVERSAL). In the code, `LedgerEntry` has only `final` fields and getters, and `BankAccount` exposes only an unmodifiable list.

(c) Keeping balance consistent with history

- Only the private `credit()` and `debit()` methods change the balance, and each one adds the ledger entry in the same step, so the two cannot drift apart.
- `isLedgerConsistent()` recomputes the balance from all entries and compares it with the stored balance (see Test 11).
- In a database, the balance update and ledger insert happen in one transaction, with periodic reconciliation.

(d) Challenges when history becomes very large

- Memory and speed: loading a huge list is slow, so use pagination and database indexes on account and date.
- Recomputing the balance from every entry is expensive, so use running balances and periodic snapshots (checkpoints).
- Storage growth: archive old entries to cheaper storage and partition by date.
- Write contention: many concurrent updates on a busy account need careful locking or queuing.

7. Observation and Results

7.1 Test cases and observations

#	Test	Expected	Observed	Result
1	Valid deposit	Balance increases	A = INR 20000.00, B = INR 3000.00	**Pass**
2	Valid withdrawal	Balance decreases	A = INR 18000.00	**Pass**
3	Zero, negative and null amounts	Rejected	`IllegalArgumentException` for all four	**Pass**
4	Overdraft limit (limit Rs 5,000)	Allowed up to -4,000; rejected beyond limit	Balance -4000.00 allowed; extra Rs 2,000 rejected	**Pass**
5	Frozen account	Withdrawal rejected; allowed after unfreeze	Rejected while FROZEN; 2900.00 after unfreeze	**Pass**
6	Closed account	Close needs zero balance; no operations afterwards	All four operations rejected as expected	**Pass**
7	Transfer Rs 5,000 (A to B)	A decreases, B increases by same amount	A: 18000 to 13000; B: 2900 to 7900	**Pass**
8	Crash after debit	Rollback; balances unchanged	A 13000 and B 7900 unchanged	**Pass**
9	Invalid transfers	Same account, closed account, over overdraft all rejected	All three rejected	**Pass**
10	Ledger immutability	History cannot be modified from outside	`UnsupportedOperationException` on `clear()`	**Pass**

#	Test	Expected	Observed	Result
11	Balance vs ledger	Always consistent	`true` for accounts A, B, C and D	**Pass**

7.2 Output screenshots (java Main)

Each test below shows the console output produced by running `java Main`.

OUTPUT --> TEST 1: Valid deposit

```
=== Test 1: Valid deposit ===
Account[ACC1001, balance=INR 20000.00, status=ACTIVE]
Account[ACC1002, balance=INR 3000.00, status=ACTIVE]
```

OUTPUT --> TEST 2: Valid withdrawal

```
=== Test 2: Valid withdrawal ===
Account[ACC1001, balance=INR 18000.00, status=ACTIVE]
```

OUTPUT --> TEST 3: Invalid amounts (zero / negative / null)

```
=== Test 3: Invalid amounts (zero / negative / null) ===
[OK]   Deposit 0 -> rejected: Transaction amount must be greater than zero
[OK]   Deposit -500 -> rejected: Transaction amount must be greater than zero
[OK]   Withdraw -100 -> rejected: Transaction amount must be greater than zero
[OK]   Withdraw null -> rejected: Transaction amount must be greater than zero
```

OUTPUT --> TEST 4: Overdraft limit

```
=== Test 4: Overdraft limit ===
Within limit : Account[ACC1003, balance=-INR 4000.00, status=ACTIVE]
[OK]   Withdraw 2000 (beyond overdraft) -> rejected: Overdraft limit exceeded. Limit: INR
5000.00, available to withdraw: INR 1000.00
```

OUTPUT --> TEST 5: Frozen account

```
=== Test 5: Frozen account ===
Account[ACC1002, balance=INR 3000.00, status=FROZEN]
[OK]   Withdraw from frozen -> rejected: Account ACC1002 is FROZEN: withdrawal not allowed
After unfreeze : Account[ACC1002, balance=INR 2900.00, status=ACTIVE]
```

OUTPUT --> TEST 6: Closed account

```
=== Test 6: Closed account ===
[OK]   Close with balance -> rejected: Account can be closed only when balance is zero
Account[ACC1004, balance=INR 0.00, status=CLOSED]
[OK]   Withdraw from closed -> rejected: Account ACC1004 is CLOSED: withdrawal not allowed
[OK]   Deposit to closed -> rejected: Account ACC1004 is CLOSED: no transactions allowed
[OK]   Unfreeze closed -> rejected: Only a FROZEN account can be unfrozen
```

OUTPUT --> TEST 7: Successful transfer of Rs 5000 (A to B)

```
=== Test 7: Successful transfer of Rs 5000 (A -> B) ===
Before: Account[ACC1001, balance=INR 18000.00, status=ACTIVE] | Account[ACC1002,
balance=INR 2900.00, status=ACTIVE]
After:  Account[ACC1001, balance=INR 13000.00, status=ACTIVE] | Account[ACC1002,
balance=INR 7900.00, status=ACTIVE]
```

OUTPUT --> TEST 8: Crash during transfer and rollback

```
=== Test 8: Crash during transfer -> rollback ===
Before: Account[ACC1001, balance=INR 13000.00, status=ACTIVE] | Account[ACC1002,
balance=INR 7900.00, status=ACTIVE]
[OK]   Transfer with crash -> rejected: Transfer TRF-002 failed and was rolled back: System
crash after debit!
After:  Account[ACC1001, balance=INR 13000.00, status=ACTIVE] | Account[ACC1002,
balance=INR 7900.00, status=ACTIVE]
A balance unchanged: true
B balance unchanged: true
```

OUTPUT --> TEST 9: Invalid transfers

```
=== Test 9: Invalid transfers ===
[OK]   Transfer to same account -> rejected: Source and destination must be different
accounts
[OK]   Transfer to closed account -> rejected: Account ACC1004 is CLOSED: no transactions
allowed
[OK]   Transfer beyond overdraft -> rejected: Overdraft limit exceeded. Limit: INR 5000.00,
available to withdraw: INR 1000.00
```

OUTPUT --> TEST 10: Ledger of Account A (immutable history)

```
=== Test 10: Ledger of Account A (immutable history) ===
TXN1001 | DEPOSIT      | INR 20000.00    | Balance: INR 20000.00   | 2026-11-11 10:50:16
TXN1003 | WITHDRAWAL   | INR 2000.00     | Balance: INR 18000.00   | 2026-11-11 10:50:16
TXN1009 | TRANSFER_OUT | INR 5000.00     | Balance: INR 13000.00   | 2026-11-11 10:50:16
TXN1011 | TRANSFER_OUT | INR 5000.00     | Balance: INR 8000.00    | 2026-11-11 10:50:16
TXN1012 | REVERSAL     | INR 5000.00     | Balance: INR 13000.00   | 2026-11-11 10:50:16
[OK]   Modify ledger directly -> rejected: UnsupportedOperationException
```

OUTPUT --> TEST 11: Balance vs ledger consistency

```
=== Test 11: Balance vs ledger consistency ===
A consistent: true
B consistent: true
C consistent: true
D consistent: true
```

8. Result

The banking application was designed and implemented in Java using self-enforcing objects. All five reported problems are prevented: invalid amounts, overdraft violations, withdrawals from frozen or closed accounts, lost money during failed transfers, and mismatch between balance and history. All eleven test cases passed.

9. Conclusion

A well-designed object protects its own invariants instead of relying on callers to follow the rules. Private fields, constructor validation, immutable value objects (`Money`), state control (`AccountStatus`), an append-only ledger and an atomic `FundTransfer` with rollback together ensure that account data stays valid, consistent and auditable. This forms the basis for the later modules on SOLID principles, failure handling and transactions.